

Rheinische Friedrich-Wilhelms-Universität Bonn

Research Module: Econometrics and Statistics

**The Mirage of Validity:
Coverage Collapse in Causal Machine Learning
under Collider Bias**

Investigating Colliders under DML and EconML

Jacob Mußler, Jorge Lobato, Varsha Rainy

Supervised by

JProf. Vladislav Morozov Ph.D.

February 13, 2026

Contents

1.	Introduction	1
2.	Literature Review	2
2.1.	High-Dimensional Causal Inference	2
2.2.	Double / Debiased Machine Learning (DML)	3
2.3.	Regression Trees, Random Forests, and Causal Forests	3
2.4.	Collider Bias and Spurious Heterogeneity	4
3.	Simulation Design & Data	5
3.1.	Benchmark DGPs	5
3.2.	Realistic DGP (WGAN)	6
3.3.	Simulation Setup	8
4.	Implementation Details - Estimators	9
4.1.	Double Machine Learning with Random Forest	10
4.2.	Causal Forest	11
5.	Results	11
5.1.	The Microscope View (Spurious Heterogeneity)	15
5.2.	Bias Analysis	16
6.	Discussion and Conclusion	17
	References	19
	A. Appendix	21

1. Introduction

Estimating causal effects, such as the impact of a policy or treatment on an outcome, is one of the central goals of econometrics. The increasing availability of high-dimensional data has expanded what researchers can study, but it has also made empirical analysis more complex. Relationships between variables are often nonlinear, the number of controls can be large, and model assumptions are easier to misspecify.

Machine learning-based approaches, including Double Machine Learning (DML) and Causal Forests, have become increasingly popular because they allow researchers to model complex relationships in a more flexible way. These methods are often presented as more robust alternatives to traditional linear models. However, their performance still depends on key identification assumptions. In particular, they are not immune to problems that arise when incorrect variables are included among the controls.

In this paper, we study what happens when a collider is included in the set of control variables. We examine how different functional forms of the collider, linear versus multiplicative, can generate spurious patterns in the data. We focus on how this affects estimation accuracy, measured by bias and mean squared error, as well as inference validity, measured by coverage probability. This leads to our main research question: *To what extent does the inclusion of collider variables induce coverage collapse in Double Machine Learning and Causal Forests, causing confidence intervals to systematically fail to reflect the true bias in treatment effect estimates?* To answer this question, we compare the performance of Double Machine Learning and Causal Forests in a controlled simulation setting.

A key concern in simulation studies is what we call the “simulation trap.” Many synthetic designs are unintentionally too simple. They may contain smooth relationships or limited structural complexity, which makes it difficult to truly challenge modern estimators. As a result, even basic linear models may appear to perform well, and conclusions may depend heavily on arbitrary parameter choices rather than genuine robustness.

To address this concern, we go beyond standard examples. In addition to conventional data-generating processes, we use a Wasserstein Generative Adversarial Network (WGAN) trained on real 401(k) data. This approach allows us to simulate data with realistic dependence structures and nonlinearities that are difficult to construct manually, so that the environments resemble applied empirical settings better.

Overall, our goal is to understand whether flexible causal machine learning methods can distin-

guish real treatment effect differences from patterns that arise purely because of inappropriate conditioning.

Section 2 reviews the literature on causal machine learning and collider bias. Section 3 describes the data-generating processes. Section 4 presents the estimation methods, including Double Machine Learning and Causal Forests, and their implementation in the simulation framework. Section 5 reports the simulation results, focusing on bias, coverage, and spurious heterogeneity. Section 6 concludes and discusses directions for future research.

2. Literature Review

2.1. High-Dimensional Causal Inference

When we talk about high dimensionality, we refer to datasets in which the number of variables p greatly exceeds the number of observations. In this context, machine learning refers to a set of statistical learning algorithms that can handle large sample sizes, a large number of variables, and unknown structural forms. Recently, the machine learning literature has expanded to focus not only on the estimation of Y , but also on the estimation of treatment effects and causal inference. However, in high-dimensional settings, this flexibility also increases the risk of conditioning on inappropriate control variables.

The literature on adapting forecasting models for inferential purposes has been significantly advanced by the contributions of Belloni, Chernozhukov, and C. Hansen (2014a) and Belloni, Chernozhukov, and C. Hansen (2014b). These papers build on the framework of Partially Linear Regression (PLR) proposed by Robinson (1988), which introduced the idea of “residualization,” and combine it with the Least Absolute Shrinkage and Selection Operator (LASSO) developed in Belloni and Chernozhukov (2013) to select valid controls for estimating both the treatment effect and the outcome.

This *post-double-selection* procedure allows for imperfect selection of controls while still delivering valid inference and well-performing confidence intervals, provided that the selected controls satisfy the identifying assumptions.

2.2. Double / Debiased Machine Learning (DML)

Double Machine Learning (DML), introduced by Chernozhukov, Chetverikov, et al. (2018), builds on the partially linear regression framework of Robinson (1988) and earlier work using LASSO for variable selection ((Chernozhukov, C. Hansen, and Spindler 2015)). The key idea behind DML is to separate the estimation of nuisance components, namely how covariates relate to the outcome and treatment, from the estimation of the causal effect itself. Instead of imposing linear models, DML allows these nuisance relationships to be learned flexibly using machine learning methods.

After estimating these relationships, DML removes their influence by residualizing both the outcome and the treatment, and then estimates the causal effect by regressing the residualized outcome on the residualized treatment. To avoid overfitting, DML uses sample splitting and cross fitting, ensuring that the data used to estimate nuisance components are separate from the data used to estimate the treatment effect.

While DML allows for complex relationships between covariates, treatment, and outcomes, it ultimately estimates a single average treatment effect. This assumption of homogeneity simplifies inference but limits DML's ability to capture meaningful differences across individuals.

2.3. Regression Trees, Random Forests, and Causal Forests

Tree based methods are popular in machine learning because they can capture nonlinear patterns and interactions in the data. Regression trees work by splitting the covariate space into regions and approximating outcomes with simple averages within each region (Breiman et al. 2017). However, individual trees tend to be unstable and sensitive to small changes in the data.

Random Forests address this issue by averaging many trees built on different subsamples, producing more stable predictions (Breiman 2001). While originally developed for prediction, these methods have been adapted for causal analysis through Causal Forests and, more generally, Generalized Random Forests (Athey, Tibshirani, and Wager 2019). Unlike DML, which targets a single global effect, Causal Forests estimate treatment effects locally, allowing them to vary across individuals.

This local flexibility is powerful, but it also comes with a risk. Because Causal Forests focus on small regions of the covariate space, they may amplify spurious patterns created by inappropriate controls. In this sense, Causal Forests can act like a microscope, highlighting local dependencies that arise from collider bias and mistaking them for real treatment effect hetero-

geneity.

2.4. Collider Bias and Spurious Heterogeneity

Although DML allows for imperfect selection of controls in the estimation of nuisance functions, this does not imply that it is immune to bias from bad controls. Indeed, the estimators obtained from DML are only as reliable as the identification and estimation of the nuisance components, meaning that a poorly tuned or ill-suited nuisance estimator can produce severely misleading results (Ahrens et al. 2025). An important source of misspecification arises when nuisance assumptions 3.1 are violated due to conditioning on a collider.

A collider is a variable that is jointly caused by two other variables, typically the treatment and the outcome variable. In directed acyclic graph (DAG) notation, a collider takes the form

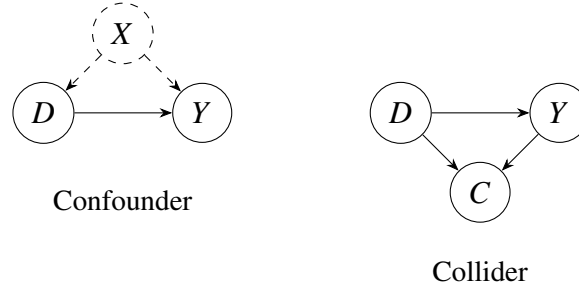


Figure 1.: Comparison of a confounder structure (left) and a collider structure (right).

where D denotes treatment and Y denotes the outcome and X denotes the confounder. Unlike confounders, colliders block paths by default and therefore do not generate bias unless they are conditioned on. However, conditioning on a collider C opens an otherwise closed backdoor path between D and Y , inducing a purely mechanical statistical dependence that does not reflect any causal relationship.

As a result, conditioning on a collider violates the unconfoundedness assumption $(Y(1), Y(0)) \perp D \mid X$, even when all true confounders are observed and included in the conditioning set. This phenomenon, known as *collider bias*, represents a class of “bad controls” in which bias is introduced through conditioning rather than omission (Elwert and Winship 2014). Importantly, collider bias arises from the causal structure of the data-generating process rather than from functional form misspecification, and it can distort both average and conditional treatment effect estimates.

In high-dimensional and nonparametric settings, where control variables are often selected automatically, colliders may be inadvertently included. This can generate spurious patterns such

as artificial treatment effect heterogeneity, reflecting induced dependence rather than genuine causal variation.

3. Simulation Design & Data

Our objective is to study the impact of including a collider as a control variable in the nuisance functions of Double Machine Learning (DML) when Random Forests are used as the underlying machine learning estimators. To this end, we construct three different data-generating processes (DGPs): *Tree-Friendly*, *PLR*, and *RealisticDGP (WGAN)*, each designed to capture different features relevant for causal estimation.

3.1. Benchmark DGPs

The Tree-Friendly DGP, inspired by the work of Fuhr, Berens, and Papies (2024), generates a treatment variable D , an outcome variable Y , and a matrix of observed covariates W . We consider $p = 11$ observed covariates $X_i \sim \mathcal{N}(0, I_p)$ and an unobserved confounder $U_i \sim \mathcal{N}(0, 1)$. The structural equation for the outcome variable is given by

$$Y_i = \tau D_i + g(X_i) + \gamma_u U_i + \varepsilon_{Yi}, \quad \varepsilon_{Yi} \sim \mathcal{N}(0, \sigma^2), \quad (1)$$

where τ represents the causal effect of the treatment on the outcome.

The treatment variable is generated according to

$$D_i = g(X_i) + \alpha_u U_i + \varepsilon_{Di}, \quad \varepsilon_{Di} \sim \mathcal{N}(0, \sigma^2). \quad (2)$$

The nuisance function $g(\cdot)$ is specified as a nonlinear function of the covariates,

$$g(X_i) = s (0.1 X_{i1}^3 + 0.5 \sin(X_{i2})), \quad (3)$$

where s controls the overall strength of confounding induced by the observed covariates. This functional form introduces smooth nonlinearities that are well approximated by tree-based methods.

Unless otherwise stated, the parameters are set to $\tau = 1.0$, $\alpha_u = 0.0$, $\gamma_u = 0.0$, $\theta = 0.0$, and the confounding strength parameter is fixed at $s = 0.2$.

The second data-generating process implements the PLR DGP used in (Chernozhukov, Chetverikov, et al. 2018). The process generates a treatment D , an outcome Y , and a set of observed covariates $X_i \in \mathbb{R}^p$, with $p = 11$.

The covariates are drawn from a multivariate normal distribution,

$$X_i \sim \mathcal{N}(0, \Sigma), \quad (4)$$

where the covariance matrix Σ has entries $\Sigma_{kj} = 0.7^{|j-k|}$, inducing a decaying correlation structure among features.

The structural equations follow a partially linear regression specification:

$$D_i = m_0(X_i) + s_1 v_i, \quad v_i \sim \mathcal{N}(0, 1), \quad (5)$$

$$Y_i = \tau D_i + g_0(X_i) + s_2 \zeta_i, \quad \zeta_i \sim \mathcal{N}(0, 1), \quad (6)$$

where τ is the true causal parameter of interest.

The nuisance functions $m_0(\cdot)$ and $g_0(\cdot)$ are nonlinear functions of the covariates and are defined as

$$m_0(X_i) = a_0 X_{i1} + a_1 \frac{\exp(X_{i3})}{1 + \exp(X_{i3})}, \quad (7)$$

$$g_0(X_i) = b_0 \frac{\exp(X_{i1})}{1 + \exp(X_{i1})} + b_1 X_{i3}, \quad (8)$$

with constants $a_0 = 1$, $a_1 = 0.25$, $b_0 = 1$, and $b_1 = 0.25$. The scaling factors s_1 and s_2 on the noise terms are set to one.

3.2. Realistic DGP (WGAN)

Because of the limitations of the two former DGPs, we introduce another DGP based on real data. To evaluate the performance of the three estimators under realistic conditions, we adopt the Wasserstein Generative Adversarial Networks (WGANs) approach proposed by Athey, Imbens, et al. (2021). We train Wasserstein Generative Adversarial Networks (WGANs) on the empirical 401(k) dataset.¹ This non-parametric approach preserves the high-dimensional correlations,

¹We utilize the `wgan` Python package which is based on the work of Athey, Imbens, et al. (2021), available at <https://github.com/gsbDBI/ds-wgan>.

non-linearities, and bounded support inherent in real economic data. The training data consists of $N = 9,915$ households from the 1991 Survey of Income and Program Participation, a standard benchmark in the causal inference literature (Chernozhukov, Chetverikov, et al. 2018). The outcome variable Y represents net financial assets, the treatment D indicates 401(k) eligibility, and the covariate vector X includes income, age, education, family size, and more. We exclude the participation variable, leaving 11 covariates, similar to the other two DGPs.

Following Athey, Imbens, et al. (2021), we make use of the decomposed joint distribution:

$$P(Y, X, D) = P(Y|X, D)P(X|D)P(D) \quad (9)$$

Rather than training a single generator for the joint distribution, we train two separate conditional generators: $G_X(Z_X, D)$ generates covariates conditional on treatment, and $G_Y(Z_Y, X, D)$ generates outcomes conditional on both. As discussed in Athey, Imbens, et al. (2021), and illustrated in Figure 2).

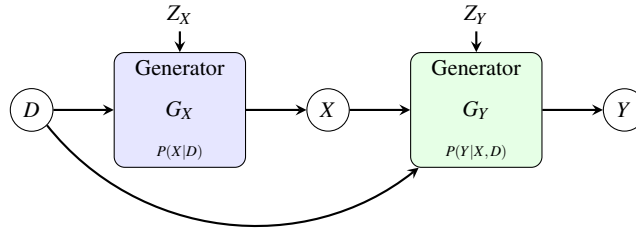


Figure 2.: Sequential Generative Architecture. G_X generates covariates conditional on treatment D ; G_Y generates outcomes conditional on X and D . Z denotes the latent noise vector.

To stabilize the training and enforce 1-Lipschitz continuity, we use a WGAN with Gradient Penalty (WGAN-GP). A specific characteristic of the utilized implementation is the use of a one-sided linear penalty for the gradient constraint, with a penalty coefficient of $\lambda = 10^2$. We employ the Adam optimizer and perform $n_{critic} = 15$ critic updates for every generator step, as a conservative setting. We implement a custom convergence criterion via an early-stopping mechanism that monitors the Wasserstein distance on a hold-out test set, with patience set to 2,000 epochs. If performance does not improve during these epochs, training stops.

Determining a ground truth for evaluation requires different approaches depending on the DGP. While the analytical Average Treatment Effect (ATE) is known for the parametric designs by

²Opposite to Gulrajani et al. 2017 and Athey, Imbens, et al. 2021, that describe a two-sided squared gradient penalty, this implementation uses a one-sided linear penalty as specified in the provided package.

construction, the WGAN relies on deep neural networks, where the true ATE cannot be derived otherwise. Consequently, we follow Athey, Imbens, et al. (2021) and compute an “Oracle ATE” using the two trained generators G_X and G_Y . We generate potential outcomes $Y^{(1)}$ and $Y^{(0)}$ for a synthetic population of $N_{pop} = 1,000,000$ units. To accommodate memory constraints, this calculation is performed in batches of 50,000, with the final ATE derived as the aggregated mean of these individual effects. To get a visual presentation of how well the synthetic data resembles the complexity of the real 401(k) dataset, refer to A.5.

3.3. Simulation Setup

To evaluate the robustness of causal estimators to collider bias, we implement a simulation framework across the DGPs presented in the sections 3.1 and 3.2. We conduct $N_{sim} = 1,000$ simulation runs for each scenario, each with a sample size of $N = 2,000$, to ensure statistical stability and to minimize the influence of sampling noise on our results.

Collider Injection For all DGPs, we consider the presence of collider variables that may be mistakenly included as controls in the nuisance functions. We study two types of colliders. The first is a multiplicative collider defined as

$$C_i = \theta(D_i Y_i) + \eta_i, \quad \eta_i \sim \mathcal{N}(0, 1). \quad (10)$$

The second is a linear collider given by

$$C_i^{\text{lin}} = \theta(0.5D_i + 0.3Y_i + 0.1X_{i1}) + v_i, \quad v_i \sim \mathcal{N}(0, 1). \quad (11)$$

By construction, both collider variables depend on the treatment and the outcome. Conditioning on these variables may therefore induce spurious associations between D_i and Y_i , allowing us to assess the consequences of including colliders as controls on the finite-sample behavior and inference properties of the estimators. We test robustness to these distortions using a strength parameter $\theta \in [0, 1]$, where $\theta = 0$ represents pure noise and $\theta \rightarrow 1$ represents a strong collider. We do that for $\theta \in \{0.0, 0.2, 0.4, 0.6, 0.8, 1.0\}$.

Evaluation Metrics We begin our assessment by evaluating the validity of inference using the 95% confidence interval Coverage Rate. To distinguish genuine robustness from artifacts driven

by variance inflation, we subsequently examine estimation accuracy via Bias and Mean Squared Error (MSE); detailed bias-variance decomposition plots are provided in the appendix A.7. Finally, we inspect Spurious Heterogeneity by plotting the estimated Conditional Average Treatment Effect (CATE) against collider values. This diagnostic detects whether non-linear estimators misattribute collider variance to treatment effect heterogeneity. For a high level overview of the performance evaluations, refer to Appendix A.8

4. Implementation Details - Estimators

We compare three estimation strategies. As a baseline, we employ Ordinary Least Squares (OLS) using the linear specification. This naive estimator serves as a benchmark to highlight the impact of misspecification when the true functional form is non-linear or when collider bias is present.

To estimate τ_0 using Double Machine Learning (DML), we employ the `DoubleMLPLR` class from the Python library `doublem1`, based on the framework developed by Chernozhukov, Chetverikov, et al. (2018). The estimator uses the observed values of D , Y , and W generated by the different DGPs considered in this study.

In order to estimate the average treatment effect τ_0 we rely on the Partially Linear Regression (PLR) framework:

$$Y = D\tau_0 + g_0(W) + U, \quad D = m_0(W) + V, \quad (12)$$

where W is a high-dimensional vector of controls, and identifying assumptions $\mathbb{E}[U|W, D] = 0$ and $\mathbb{E}[V|W] = 0$ hold. It is important to note that W represents the set of conditioning variables available to the estimator: in our baseline scenarios, W consists solely of the structural covariates ($W = X$), whereas in the collider scenarios, it also includes the collider variable ($W = \{X, C\}$). We employ the Double Machine Learning (DML) "partialling out" procedure to isolate the variation in D that is orthogonal to W . This involves three steps: (1) estimating the conditional expectations $l_0(W) = \mathbb{E}[Y|W]$ and $m_0(W) = \mathbb{E}[D|W]$ using flexible machine learning learners; (2) computing the residuals $\tilde{Y} = Y - \hat{l}_0(W)$ and $\tilde{D} = D - \hat{m}_0(W)$; and (3) regressing $\tilde{Y}$ on $\tilde{D}$ to obtain τ_0 . This orthogonalization ensures that the estimation error in the nuisance functions affects the parameter of interest only at a second-order magnitude (see Appendix A.2 for the formal derivation and score function decomposition).

4.1. Double Machine Learning with Random Forest

We estimate the nuisance functions need it for DML using Random Forests implemented via the `RandomForestRegressor` class from the Python library `sklearn.ensemble`³. Specifically, we obtain $\hat{l}_0(W)$ by fitting a Random Forest regression of Y on W , and $\hat{m}_0(W)$ by fitting a Random Forest regression of D on W . For a more detailed explanation of how Random Forests operate, see Appendix A.1 or B. Hansen (2022).

To prevent overfitting and preserve orthogonality between the nuisance estimation and the target parameter, we implement cross-fitting using the DML2 aggregation method.

Sample Splitting: Let N denote the total sample size and K the number of folds ($K = 5$ in our implementation). The sample is partitioned into K mutually exclusive subsets $I_1, \dots, I_K$. For each fold k , the nuisance functions $\hat{l}_{0,k}$ and $\hat{m}_{0,k}$ are trained on the auxiliary sample $I_k^c = \{1, \dots, N\} \setminus I_k$, where $n = |I_k| = N/K$ represents the size of each fold.

Estimator Construction: Define the true and estimated residualized treatment as

$$V_i = D_i - m_0(W_i), \quad \hat{V}_i = D_i - \hat{m}_{0,k}(W_i). \quad (13)$$

The final estimator is obtained using the DML2 algorithm, which pools the residuals (scores) from all K folds and solves the moment condition globally. This approach is often more stable in finite samples as it avoids issues with small denominators in individual folds. The estimator is given by:

$$\tilde{\tau}_0 = \left(\sum_{k=1}^K \sum_{i \in I_k} \hat{V}_i^2 \right)^{-1} \sum_{k=1}^K \sum_{i \in I_k} \hat{V}_i (Y_i - \hat{l}_{0,k}(W_i)). \quad (14)$$

We analyze the estimation error to illustrate how DML achieves $\sqrt{N}$ -consistency despite relying on potentially slow-converging Machine Learning estimators for the nuisance functions.

Asymptotic Properties and Neyman Orthogonality To ensure valid statistical inference, DML exploits the specific structure of the score function to attain $\sqrt{N}$ -consistency even when the nuisance estimators $\hat{l}_0$ and $\hat{m}_0$ converge at slower rates (e.g., $N^{-1/4}$).

The key property underlying this robustness is **Neyman Orthogonality**. This property guaran-

³Hyperparameters for the nuisance functions $l_0(W)$ and $m_0(W)$ were optimized in a separate tuning script using Optuna Akiba et al. 2019 to ensure high-quality nuisance estimation while maintaining computational feasibility. See Appendix A.1 for the specific configurations used for each DGP.

tees that small estimation errors in the nuisance components affect the estimation of τ_0 only at second order. As shown in Chernozhukov, Chetverikov, et al. (2018), the scaled estimation error can be decomposed into a leading stochastic term, a regularization bias, and an overfitting bias. Cross-fitting eliminates the overfitting bias, while the product structure of the regularization bias ensures that it vanishes asymptotically under mild rate conditions.

For the complete mathematical decomposition of the score function, the bias components (a^*, b^*, c^*) , and the formal proof of asymptotic normality, see Appendix A.3.

4.2. Causal Forest

To estimate τ using Causal Forests, we employ the `CausalForestDML` class from the Python library `econml.dml`. The estimation procedure follows the same logic described above up to Step 2. That is, we assume a partially linear model, estimate the nuisance functions using Random Forests, and compute the corresponding residuals.

The key difference lies in the final stage. Instead of estimating a constant treatment effect, Causal Forests allow for heterogeneous treatment effects by learning a flexible, nonparametric mapping from W to treatment effects. This enables the estimation of conditional average treatment effects (CATEs).

In our context, the purpose of using this approach is to evaluate whether the inclusion of a collider induces spurious heterogeneity in the estimated treatment effects. For a more detailed discussion of Causal Forests and their theoretical properties, see Appendix A.4 or Athey, Tibshirani, and Wager (2019).

5. Results

Figure 3 shows the distribution of the estimated $\hat{\tau}$ for the naive, multiplicative collider ($\theta = 1$), and linear collider ($\theta = 1$) scenarios, under OLS, DoubleML and EconML for the TF, PLR and WGAN DGPs.

For all cases under TF and PLR, we observe that in the naive scenario the distribution is centered around the true $\hat{\tau}$. This shows that, in the absence of collider bias, all models estimate on average the correct parameter. However, once we include the collider (both multiplicative and linear), the distributions become centered around an incorrect $\hat{\tau}$. This indicates higher variability and systematic bias: the estimators converge to a pseudo-true parameter, meaning

they are consistent, but consistent to the wrong value due to the violation of the identification assumptions.

This effect is particularly interesting in the case of EconML. Since it estimates heterogeneous effects, the model assigns weights to observations depending on the learned structure. If the collider bias is correlated with covariates used by the model, EconML can amplify those distorted patterns. In this sense, the same flexibility that allows the model to capture heterogeneity also makes it more sensitive to structural distortions such as collider bias.

Additionally, when analyzing TF and PLR more carefully, we notice that OLS and DoubleML appear to perform well even under the inclusion of a collider. At first glance, one might conclude that these estimators are robust to collider bias. Nevertheless, this apparent robustness is driven by weaknesses in the DGP rather than by true estimator strength.

In the case of TF, the results reveal what we call a “Noise Masking” fallacy. The variance of the error term is relatively large compared to the non-linear signal. As a consequence, the non-linear component becomes buried in the noise, and the models minimize the loss by fitting an approximately straight line. The issue is not that the model correctly captures the true structure, but that the non-linear distortion is not strong enough to be identified relative to the noise level. This explains why OLS and DoubleML under the multiplicative scenario converge to values centered close to the true parameter: the distortion is present, but effectively hidden. For a detailed view of the “Noise Masking” fallacy, please refer to Appendix A.6.

For the PLR case, the results illustrate a “Lucky Linearity” fallacy. The sigmoid confounding function $\frac{1}{1+e^{-x}}$ is approximately linear for values close to zero. Since the covariates follow $X \sim \mathcal{N}(0, 1)$, most observations lie near zero, meaning that estimation occurs in a region where the function behaves almost linearly. This local linearity allows OLS to perform well by accident, even though the DGP is not linear. The good performance is therefore not due to correct model specification, but to the data concentrating in a region where the violation is weak. For a detailed view of the “Lucky Linearity” fallacy, please refer to Appendix A.6.

Overall, these results show that the simulated DGPs (TF and PLR) are prone to “lucky” simplifications that create the misleading impression that the models are robust to collider bias. The estimators do not truly solve the identification problem; rather, the structure of the DGP reduces the visible impact of the bias.

To evaluate the estimators in a more realistic and complex environment, we test them under the WGAN DGP. The complexity of the learned structure makes the bias more evident, allow-

ing us to better assess whether the estimators are genuinely robust rather than benefiting from accidental simplifications.

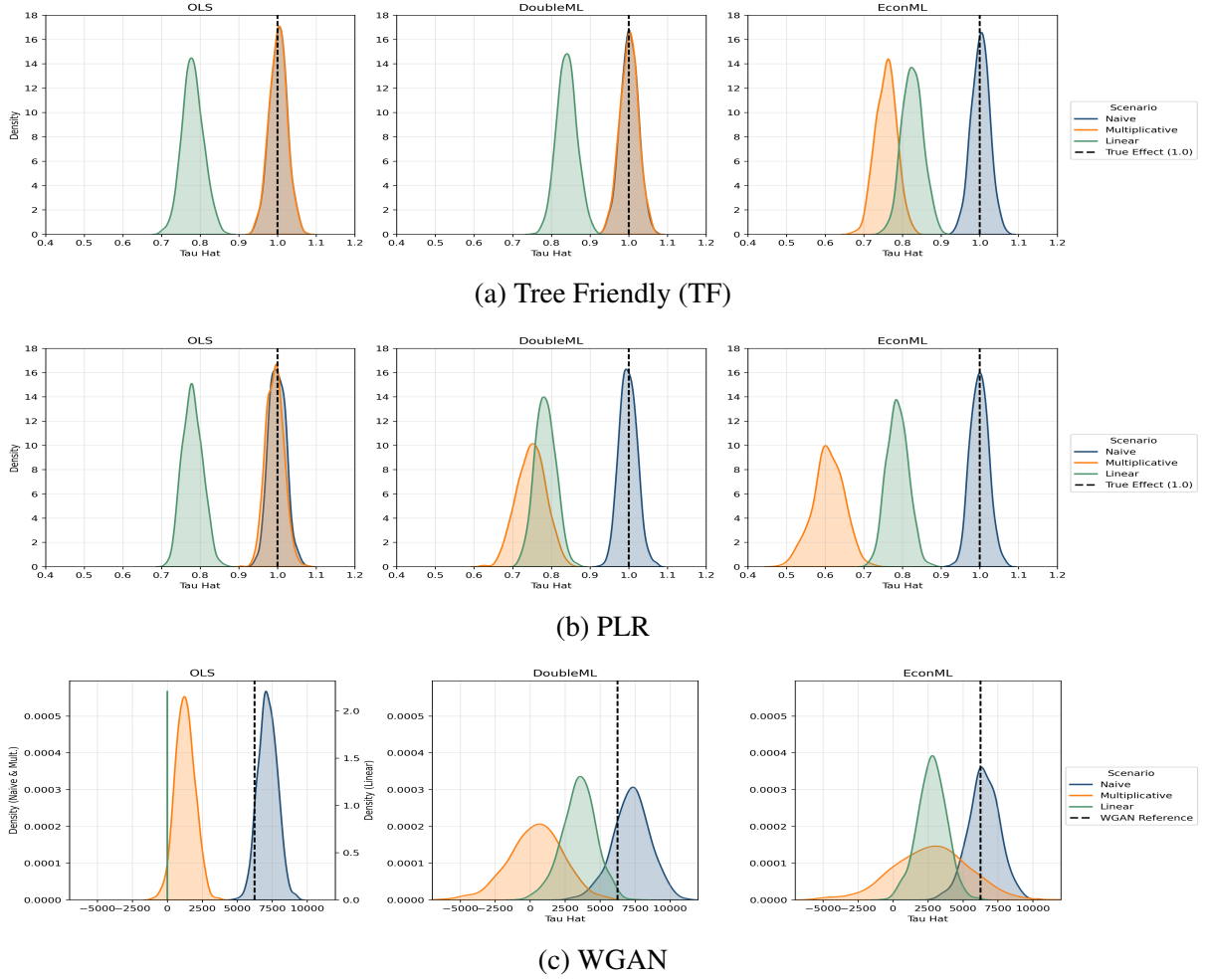


Figure 3.: Distribution of Estimates for TF, PLR, and WGAN methods.

For WGAN, the data structure is considerably more complex. Even in the naive scenario, the estimated $\hat{\tau}$ are not centered around the true parameter for OLS and DoubleML. This suggests that, under complex data structures, linear approximations are not sufficient to recover the correct causal parameter. In contrast, EconML shows a better approximation to the correct $\hat{\tau}$, consistent with the fact that the WGAN DGP contains heterogeneous treatment effects that can be captured by this estimator.

For OLS, we clearly notice that the estimator breaks down. Under the linear collider, it converges to an incorrect estimated $\hat{\tau}$, revealing strong structural bias. In the multiplicative case, the collider spreads the distribution of $\hat{\tau}$, generating a wider range of values.

For DoubleML and EconML, the distributions appear more stable despite the inclusion of collider bias, especially compared to OLS. Even if the variance is better controlled, both estimators

are centered around an incorrect $\hat{\tau}$. This indicates that once the collider is included, the conditional independence assumptions required for causal identification are violated. As a consequence, the estimators converge systematically to a biased pseudo-true parameter. For a detailed view of the estimator performance, please refer to Appendix A.8.

In contrast with TF and PLR, the WGAN DGP provides a more realistic and demanding evaluation of the estimators. The absence of “lucky” linear regions or weak non-linear signals prevents the masking of distortions. Therefore, the limitations of each estimator become more evident. The results show that the apparent robustness observed in simpler DGPs does not generalize to more complex environments. Under realistic data-generating processes, collider bias produces systematic distortions that no estimator can correct without restoring the identification conditions.

The coverage rates represent the probability that the estimated treatment effect lies within the true effect. Figure 4 compares coverage across Linear and Multiplicative scenarios for different DGPs, estimators, and values of the collider strength. All estimators exhibit stable and close-to-one coverage when the collider effect is weak. However, as the strength increases, coverage falls, indicating a loss of inferential validity. In most settings, OLS is the first estimator to experience a decline in coverage, showing its sensitivity to model misspecification. DoubleML and EconML display greater robustness in both Linear and Multiplicative cases; however, their performance also deteriorates as collider strength increases. Under the WGAN DGP, coverage is low even at low values of θ , suggesting that the complexity of this data-generating process presents challenges for all methods. These results show that even though machine-learning models improve robustness relative to OLS, correct inference remains difficult in highly complex scenarios with collider bias.

An important result from the coverage rates is that OLS appears to perform very well and is not affected by the strength of the collider in the multiplicative scenario for TF and PLR, presenting coverage close to 1 in almost all cases. A similar pattern is observed for EconML under the WGAN DGP. At first glance, this could give the impression that these estimators are robust to the presence of a collider, even in a complex DGP.

However, it is important to consider that coverage does not depend only on the Mean Squared Error, but also on the variance of the estimator. Coverage is directly related to the width of the confidence interval. If an estimator exhibits large variance, the confidence interval becomes wider. As a consequence, the range within which the true parameter may fall increases, raising

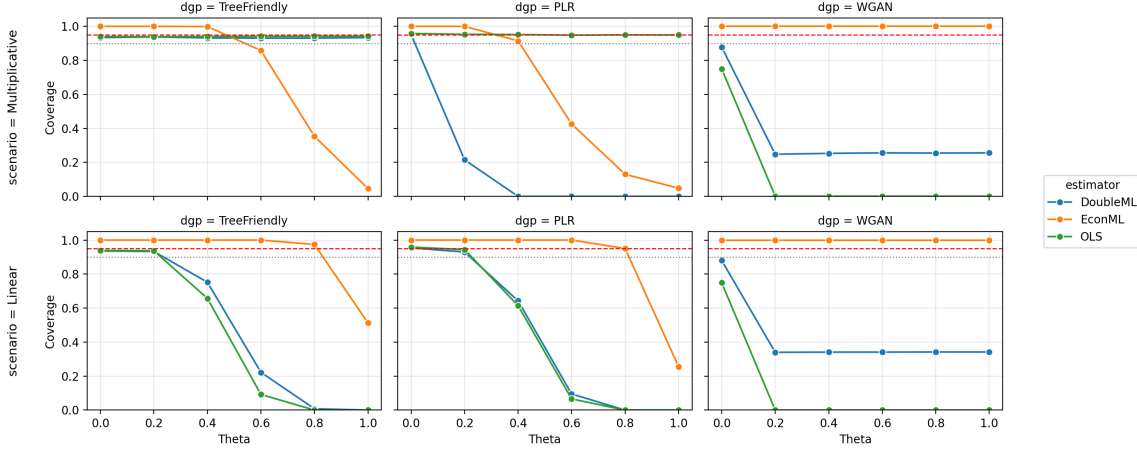


Figure 4.: Coverage Rates Comparison: TreeFriendly, PLR and WGAN

the probability that the estimator covers the true value. Therefore, a coverage rate close to one does not necessarily imply that the estimator is unbiased or stable.

In this case, high coverage may be driven by inflated variance rather than genuine robustness. This seems to occur with OLS in the multiplicative scenario for TF and PLR, and with EconML under WGAN. Although coverage remains high, the estimator may still suffer from instability or large dispersion in the estimated treatment effects. In other words, the model “covers” the true parameter frequently, partly because the confidence intervals are sufficiently wide to compensate for the variability.

This result highlights that coverage should always be interpreted jointly with Bias and Variance. Looking at coverage alone may lead to misleading conclusions about estimator performance. For an examination of the Bias and Variance decomposition please refer to Appendix A.7 and Appendix A.8.

5.1. The Microscope View (Spurious Heterogeneity)

To analyze the effect of collider bias on the estimation of the treatment effects under the EconML estimator, we construct a “Microscope view.” The purpose of this figure is to observe how the value of the collider affects the estimation of $\hat{\tau}$, and more specifically, to evaluate whether the inclusion of the collider generates spurious heterogeneity, in other words, heterogeneous treatment effects even when the true treatment effect is constant.

We focus only on the multiplicative collider case, since it was the scenario where the collider showed the most relevant pattern in the previous results. By concentrating on this case, we can

more clearly identify the mechanism through which the collider distorts the estimation of the treatment effect.

In the case of the TF DGP, we observe a very clear pattern. When the collider value is low, the estimator tends to underestimate the treatment effect. As the collider value increases, the estimator starts to overestimate it. This creates a systematic relationship between the collider and the estimated treatment effect. Since the true treatment effect is constant in this setting, this pattern cannot be interpreted as real heterogeneity. Instead, it reflects that the model is attributing variation in the collider to variation in the treatment effect, which leads to the creation of artificial or spurious heterogeneity.

For the PLR DGP, we observe a similar behavior, although the pattern is less sharp and not as visually strong as in the TF case. The dispersion is higher and the relationship is noisier, but there is still a visible dependence between the collider value and the estimated treatment effect. This again suggests that the inclusion of the collider induces heterogeneity that is not present in the true data-generating process. Even if the effect is less pronounced, the mechanism appears to be similar.

For the WGAN DGP, we plot the estimated CATE centered by the global ATE. The results show that most of the values are concentrated close to zero. This is consistent with the fact that WGAN generates genuinely heterogeneous treatment effects, and EconML is designed to capture such heterogeneity.

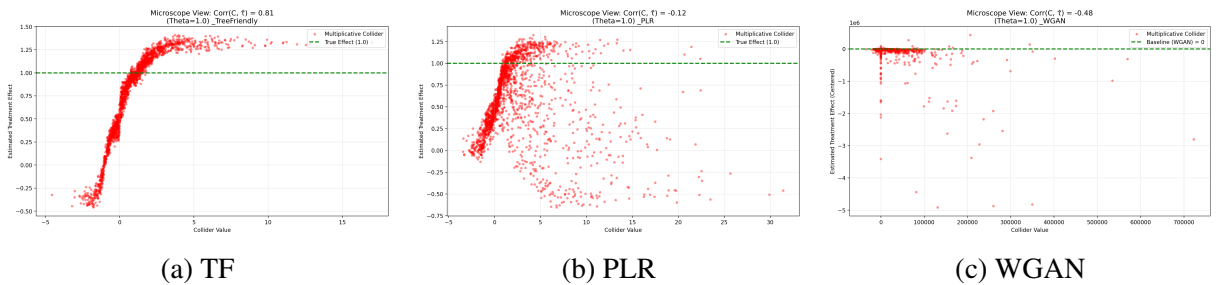


Figure 5.: The Microscope Effect across TF, PLR, and WGAN methods.

5.2. Bias Analysis

Regarding the bias, we can observe that, in general, as the strength of the collider increases, the level of the bias also increases. This pattern is consistent across most DGPs and estimators, showing that the presence of a stronger collider systematically distorts the estimation of the treatment effect.

In general, across the different scenarios, OLS is the estimator that starts to deteriorate first as the collider strength increases. The bias grows more rapidly compared to DoubleML and EconML, indicating that OLS is more sensitive to model misspecification and to the inclusion of colliders. In the case of the WGAN DGP, all estimators exhibit a sharp increase in bias even for relatively low values of the collider strength. This suggests that when the data-generating process is already complex, the presence of a collider amplifies the estimation errors more quickly.

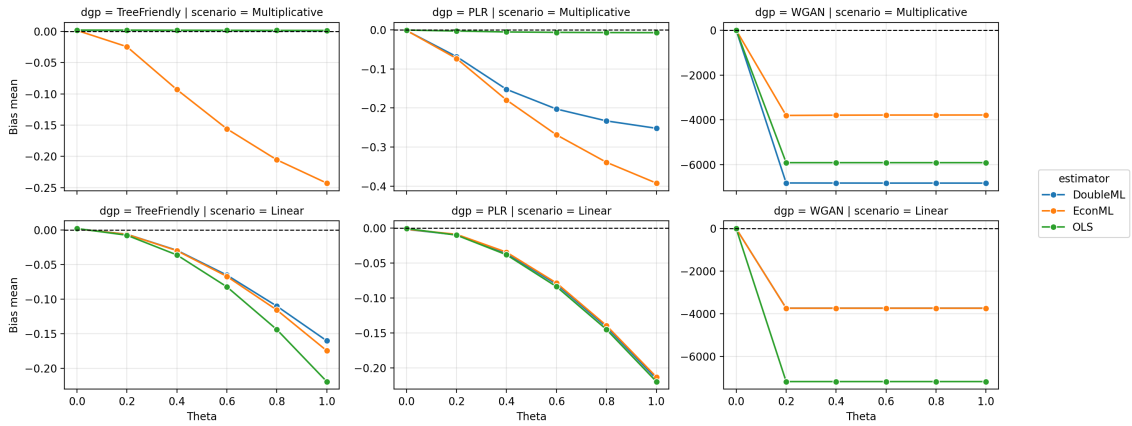


Figure 6.: Bias Comparison: PLR, TreeFriendly and WGAN

An interesting result is that OLS presents a value of zero bias in the Multiplicative case, even when the collider strength increases. At first glance, this may suggest that OLS is not affected in this scenario. However, this is directly related to the fallacy case that we discussed previously. The zero bias does not necessarily imply correct model specification or robustness. Instead, it reflects a specific structure of the multiplicative collider in which the bias cancels out at the mean level. This means that, although the average bias is zero, the estimator may still suffer from instability or other distortions, as we observed in the variance and coverage analysis.

6. Discussion and Conclusion

Even though DML and Causal Forest present the advantage of not requiring a perfectly specified nuisance function or a fully correct set of control variables, this does not imply that these models are strong enough to detect collider bias and automatically exclude it from the model. In other words, flexibility in nuisance estimation does not guarantee robustness to incorrect variable inclusion.

The results show that as the presence and strength of the collider increase, the correct estimation of DML and Causal Forest starts to deteriorate, gradually converging to an incorrectly estimated value. This situation becomes even more severe in scenarios with realistic and complex data-generating processes. The inclusion of a collider not only increases bias, but also affects coverage rates, which begin to fail in reflecting the true distortion in the estimates. Moreover, coverage can be sensitive to the interaction between variance and Mean Squared Error, potentially leading to incorrect interpretations of estimator performance.

In direct response to the research question, to what extent does the inclusion of collider variables induce coverage collapse in Double Machine Learning and Causal Forests?, the evidence suggests that collider inclusion can indeed generate coverage collapse, especially as collider strength increases and in more complex DGPs. Confidence intervals may systematically fail to reflect the true bias in treatment effect estimates, either because bias increases substantially or because variance inflates the interval width, masking underlying distortions. Therefore, coverage alone is not a sufficient diagnostic of robustness.

Moreover, This paper highlights the importance of testing these models under realistic data conditions. Evaluating estimators only under simplified settings can lead to incorrect conclusions about their performance. When applied to more complex environments, such as those represented by the WGAN DGP, weaknesses become more evident.

Another important finding is that the inclusion of a multiplicative collider creates spurious heterogeneity. Even when the true treatment effect is constant, the model can generate spurious heterogeneity by attributing variation in the collider to variation in the estimated treatment effect. This reinforces the idea that flexible machine learning methods can misinterpret structural distortions as genuine heterogeneity.

Finally, this paper had to reduce the number of observations and covariates in the DGPs due to time and resource constraints. However, this limitation opens an opportunity for future research to test these models under more complex scenarios, larger samples, and more demanding robustness checks.

References

- Ahrens, Achim et al. (2025). *An Introduction to Double/Debiased Machine Learning*. arXiv: 2504.08324 [econ.EM]. URL: <https://arxiv.org/abs/2504.08324>.
- Akiba, Takuya et al. (2019). “Optuna: A next-generation hyperparameter optimization framework”. In: *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pp. 2623–2631.
- Athey, Susan, Guido W Imbens, et al. (2021). “Using Wasserstein Generative Adversarial Networks for the Design of Monte Carlo Simulations”. In: *Journal of Econometrics* 221.2, pp. 836–857.
- Athey, Susan, Julie Tibshirani, and Stefan Wager (Apr. 2019). “Generalized Random Forests”. In: *The Annals of Statistics* 47.2, pp. 1148–1178.
- Belloni, Alexandre and Victor Chernozhukov (2013). “Least squares after model selection in high-dimensional sparse models”. In: *Bernoulli* 19.2, pp. 521–547. DOI: 10.3150/11-BEJ410. URL: <https://doi.org/10.3150/11-BEJ410>.
- Belloni, Alexandre, Victor Chernozhukov, and Christian Hansen (2014a). “High-Dimensional Methods and Inference on Structural and Treatment Effects”. In: *Journal of Economic Perspectives* 28.2, pp. 29–50. DOI: 10.1257/jep.28.2.29.
- (2014b). “Inference on Treatment Effects after Selection among High-Dimensional Controls”. In: *Review of Economic Studies* 81.2, pp. 608–650. DOI: 10.1093/restud/rdt044.
- Breiman, Leo (2001). “Random forests”. In: *Machine learning* 45.1, pp. 5–32.
- Breiman, Leo et al. (2017). *Classification and regression trees*. Chapman and Hall/CRC.
- Chernozhukov, Victor, Denis Chetverikov, et al. (2018). “Double/Debiased Machine Learning for Treatment and Structural Parameters”. In: *The Econometrics Journal* 21.1, pp. C1–C68.
- Chernozhukov, Victor, Christian Hansen, and Martin Spindler (2015). “Post-Selection and Post-Regularization Inference in Linear Models with Many Controls and Instruments”. In: *American Economic Review: Papers and Proceedings* 105.5, pp. 486–490.
- Elwert, Felix and Christopher Winship (2014). “Endogenous Selection Bias: The Problem of Conditioning on a Collider Variable”. In: *Annual Review of Sociology* 40, pp. 31–53.
- Fuhr, Jonathan, Philipp Berens, and Dominik Papies (2024). *Estimating Causal Effects with Double Machine Learning – A Method Evaluation*. arXiv: 2403.14385 [stat.ML]. URL: <https://arxiv.org/abs/2403.14385>.

- Gulrajani, Ishaan et al. (2017). “Improved training of wasserstein gans”. In: *Advances in neural information processing systems*. Vol. 30.
- Hansen, Bruce (2022). *Econometrics*. Princeton University Press.
- Robinson, Peter M. (1988). “Root-N-Consistent Semiparametric Regression”. In: *Econometrica* 56.4, pp. 931–954.

A. Appendix

A.1. Random Forest

Random Forests are built using many regression trees, which are an algorithm that estimates conditional mean functions of the form

$$m(w) = \mathbb{E}[Y \mid W = w]. \quad (1)$$

This structure allows Random Forests to flexibly approximate the nuisance functions required in the DML framework.

The covariate space $\mathcal{W}$ is partitioned into disjoint regions, and the outcome variable is approximated by a constant within each region. As a result, the estimated regression function takes the form of a multidimensional step function.

The estimation proceeds through a recursive sample-splitting algorithm, which consists of a sequence of threshold regressions. At each split, the regression sample-splitting algorithm is applied:

$$Y = \mu_1 \mathbf{1}\{W_d \leq \gamma\} + \mu_2 \mathbf{1}\{W_d > \gamma\} + e, \quad (2)$$

where d indexes the covariate used for splitting, γ denotes the splitting threshold, and the error term satisfies $\mathbb{E}[e \mid W] = 0$. The algorithm selects the split that reduces the sum of squared residuals, subject to a chosen minimum leaf size constraint, and continues splitting until no further admissible splits remain.

The terminal nodes define regions in which the conditional mean is estimated by the sample average of the outcomes. The model is called regression trees, since each time a subsample is created using the splitting algorithm, a branch is formed, and when a branch cannot be split anymore (that is, it becomes a terminal node), a leaf is created. In this way, the model takes the form of a decision tree.

A Random Forest estimator is obtained by averaging a large number of regression trees grown using bootstrap resamples of the original data; this procedure is called bagging. In addition, at each split, a random subset of covariates is considered, which helps to reduce the correlation between trees and leads to a reduction in variance.

The algorithm for Random Forests is as follows:

1. First, we select a minimal number of observations in each leaf $N_{\min}$, a minimal number of observations required to make a split, and a sampling number for the covariates considered in each split.
2. For each tree $b = 1, \dots, B$:
 - (a) We create a nonparametric bootstrap sample.
 - (b) With the bootstrap sample that we create, we construct a regression tree. First, we select m variables from the p regressors. Then, using the m variables, we choose the one that produces the best regression split. Each split is required to have at least $N_{\min}$ observations in each leaf, and we split the bootstrap sample fulfilling these conditions.
 - (c) We stop when no further admissible splits remain, and we set $\hat{m}_b(x)$ as the sample mean of the outcome variable in each leaf of the tree.
3. We estimate the Random Forest predictor as

$$\hat{m}_{\text{rf}}(x) = B^{-1} \sum_{b=1}^B \hat{m}_b(x), \quad (3)$$

where B denotes the total number of regression trees, b indexes the individual trees, and $\hat{m}_b(x)$ denotes the prediction of tree b at the point x . The same averaging procedure is used to obtain both nuisance estimators $\hat{l}_0(x)$ and $\hat{m}_0(x)$, depending on whether the outcome in the regression tree is Y or D .

To ensure optimal performance, the hyperparameters B (n_estimators), $N_{\min}$ (min_samples_leaf), and m (max_features) were tuned separately for each DGP using Optuna. The specific values used in the simulation are reported in Table 1.

Table 1.: Hyperparameters for Random Forest Nuisance Estimators

DGP	Trees (B)	Features (m)	Min Leaf (N_{min})	Min Split	Max Depth
Tree-Friendly	500	30%	6	5	5
PLR	400	80%	15	7	None
WGAN	400	$\sqrt{p}$	17	14	50

A.2. Derivation of the DML Estimator

In the following, we describe the key steps underlying the construction of this estimator.

Step 1: Conditional Expectation. Taking expectations conditional on W ,

$$\mathbb{E}[Y | W] = \mathbb{E}[D\tau_0 + g_0(W) + U | W] = m_0(W)\tau_0 + g_0(W) =: l_0(W). \quad (4)$$

Step 2: Partialization. Subtracting $l_0(W)$ from the outcome equation yields

$$Y - l_0(W) = (D - m_0(W))\tau_0 + U. \quad (5)$$

Defining the residuals $\tilde{Y} := Y - l_0(W)$ and $\tilde{D} := D - m_0(W) = V$, we obtain

$$\tilde{Y} = \tilde{D}\tau_0 + U. \quad (6)$$

This transformation removes the influence of the high-dimensional controls W , isolating the variation in D that is orthogonal to W .

Step 3: Identification. Multiplying by V and taking expectations,

$$\mathbb{E}[V\tilde{Y}] = \mathbb{E}[V^2]\tau_0, \quad (7)$$

where $\mathbb{E}[VU] = 0$. Under $\mathbb{E}[V^2] > 0$,

$$\tau_0 = (\mathbb{E}[V^2])^{-1}\mathbb{E}[V\tilde{Y}]. \quad (8)$$

In practice, l_0 and m_0 are unknown and are estimated using Machine Learning methods, yielding $\hat{l}_0$ and $\hat{m}_0$. In this project, we use Random Forests and Causal Forests to estimate these nuisance functions.

A.3. Asymptotic Analysis of the DML Estimator

In this section, we provide the detailed decomposition of the Double Machine Learning (DML) estimator's error terms, following Chernozhukov, Chetverikov, et al. (2018).

Using the score function

$$\psi(W; \tau, \eta) = (Y - l(W) - \tau(D - m(W)))(D - m(W)), \quad (9)$$

the scaled estimation error can be decomposed as follows (Chernozhukov, Chetverikov, et al. 2018):

$$\sqrt{N}(\tilde{\tau}_0 - \tau_0) = \underbrace{a^*}_{\text{Noise}} + \underbrace{b^*}_{\text{Regularization Bias}} + \underbrace{c^*}_{\text{Overfitting Bias}}. \quad (10)$$

A. The Noise Term (a^*)

$$a^* = (\mathbb{E}[V^2])^{-1} \frac{1}{\sqrt{N}} \sum_{i=1}^N V_i U_i. \quad (11)$$

This term represents irreducible statistical noise. It obeys the Central Limit Theorem and converges to a normal distribution $\mathcal{N}(0, \Sigma)$.

B. The Regularization Bias Term (b^*)

$$b^* = (\mathbb{E}[V^2])^{-1} \frac{1}{\sqrt{N}} \sum_{k=1}^K \sum_{i \in I_k} (\hat{m}_{0,k}(W_i) - m_0(W_i)) (\hat{l}_{0,k}(W_i) - l_0(W_i)). \quad (12)$$

This term captures the product of the estimation errors of the nuisance functions. While individual Machine Learning estimators may converge slowly (e.g., at rate $N^{-1/4}$), the bias depends on their product. As a result, if both nuisance estimators converge at rate $N^{-1/4}$, the product converges at rate $N^{-1/2}$, making b^* asymptotically negligible.

This result follows from the **Neyman Orthogonality** of the score function. In particular, the Gateaux derivative with respect to the nuisance parameter η vanishes at the true value η_0 :

$$\partial_\eta \mathbb{E}[\psi(W; \tau_0, \eta_0)][\eta - \eta_0] = 0. \quad (13)$$

C. The Overfitting Bias Term (c^*)

$$c^* = (\mathbb{E}[V^2])^{-1} \frac{1}{\sqrt{N}} \sum_{k=1}^K \sum_{i \in I_k} V_i (\hat{l}_{0,k}(W_i) - l_0(W_i)). \quad (14)$$

Without sample splitting, the noise term V_i would be correlated with the estimation error of $\hat{l}_0$, since both would be computed on the same data. By using cross-fitting, $\hat{l}_{0,k}$ is trained on I_k^c and is independent of V_i for $i \in I_k$, implying that this term vanishes in probability.

Since the bias terms b^* and c^* vanish, the estimator behaves asymptotically as if the true nuisance functions were known:

$$\sigma^{-1}\sqrt{N}(\tilde{\tau}_0 - \tau_0) \xrightarrow{d} \mathcal{N}(0, 1). \quad (15)$$

A.4. Causal Forest

When working with Causal Forests, we adopt a more flexible framework in which the target parameter is allowed to vary with the covariates. In particular, we estimate the conditional treatment effect $\tau(w)$ as the solution to a *local estimating equation*:

$$\mathbb{E}[\psi_{\tau(w), v(w)}(O_i) \mid W_i = w] = 0, \quad \text{for all } w \in \mathcal{W}, \quad (16)$$

where $\psi(\cdot)$ denotes a problem-specific score function, $v(w)$ represents an optional nuisance parameter, and $O_i = (Y_i, D_i, W_i)$ collects the observed data.

In the context of the partially linear model introduced in the main text, the parameter $\tau(w)$ corresponds to the Conditional Average Treatment Effect (CATE). When the treatment effect is constant, $\tau(w) = \tau_0$ for all w , and therefore the heterogeneous framework nests the constant-effect case.

As in the DoubleML procedure, the score function $\psi_\tau(\cdot)$ is constructed after residualizing the outcome and treatment variables with respect to the nuisance functions $l_0(W)$ and $m_0(W)$.

Defining the residualized variables

$$\tilde{Y}_i = Y_i - l_0(W_i), \quad \tilde{D}_i = D_i - m_0(W_i), \quad (17)$$

the orthogonal score for $\tau(w)$ is given by

$$\psi_\tau(O_i) = \tilde{D}_i (\tilde{Y}_i - \tau(W_i) \tilde{D}_i). \quad (18)$$

The Causal Forest estimator is then defined as the solution to the following weighted empirical estimating equation:

$$(\hat{\tau}(w), \hat{v}(w)) \in \arg \min_{\tau, v} \left| \sum_{i=1}^n \alpha_i(w) \psi_{\tau, v}(O_i) \right|^2, \quad (19)$$

where the weights $\alpha_i(w)$ are obtained from a Random Forest and capture how often observation i falls in the same terminal leaf as a target point w across the ensemble. More precisely, for each tree $b = 1, \dots, B$,

$$\alpha_{bi}(w) = \frac{\mathbf{1}\{W_i \in L_b(w)\}}{|L_b(w)|}, \quad \alpha_i(w) = \frac{1}{B} \sum_{b=1}^B \alpha_{bi}(w), \quad (20)$$

where $L_b(w)$ denotes the terminal leaf of tree b containing w . These weights define a data-adaptive neighborhood around w , allowing Causal Forests to estimate $\tau(w)$ using observations that are locally relevant in the covariate space.

In practice, Causal Forests implement sample splitting and honesty, ensuring that the data used to construct the forest structure are independent from the data used to estimate treatment effects within each leaf. This preserves orthogonality and supports valid statistical inference.

Step 3: Solving for $\tau(w)$

Given the weighted score equation, the local treatment effect is estimated by

$$\hat{\tau}(w) = \frac{\sum_{i=1}^n \alpha_i(w) \tilde{D}_i \tilde{Y}_i}{\sum_{i=1}^n \alpha_i(w) \tilde{D}_i^2}. \quad (21)$$

This expression is the solution to the weighted orthogonal moment condition, where the score function satisfies

$$\psi_{\tau}(O_i) = \tilde{D}_i (\tilde{Y}_i - \tau(W_i) \tilde{D}_i).$$

This representation makes explicit the connection with the DML estimator presented in the main text. The key difference is that the moment condition is now solved locally using the forest weights $\alpha_i(w)$, which allows for heterogeneous treatment effects.

A.5. WGAN Validation

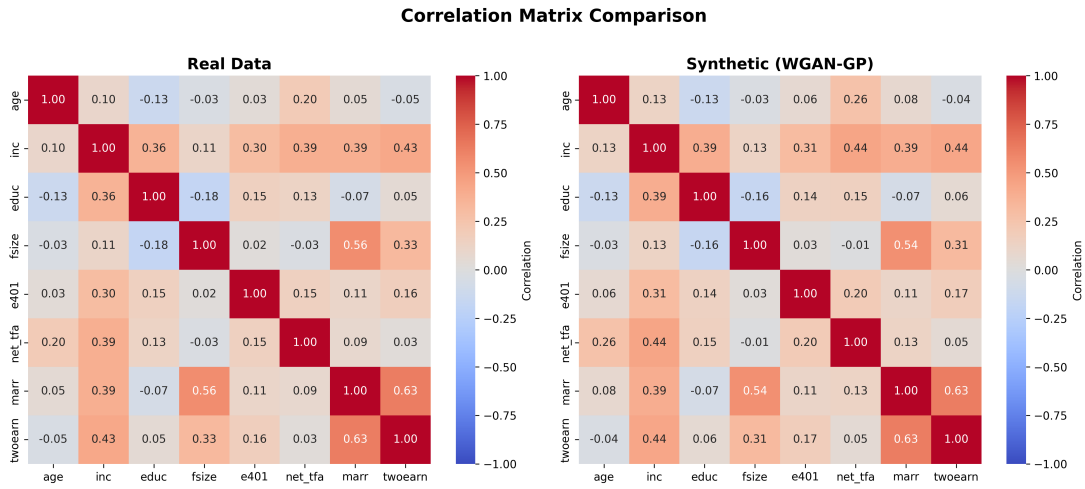


Figure 7.: WGAN Correlation Comparison. Further supplementary figures can be found in the associated GitHub repository.

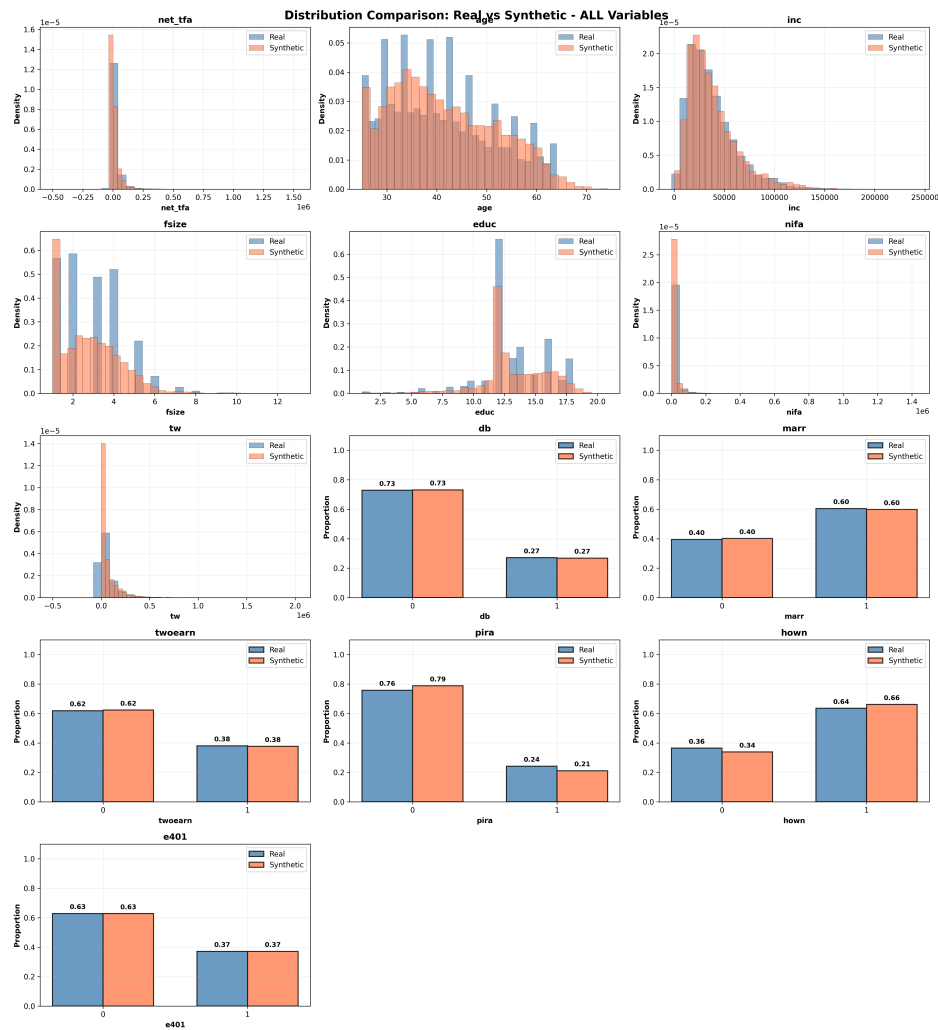


Figure 8.: WGAN Distribution Comparison. Further supplementary figures can be found in the associated GitHub repository.

A.6. Fallacy Analysis

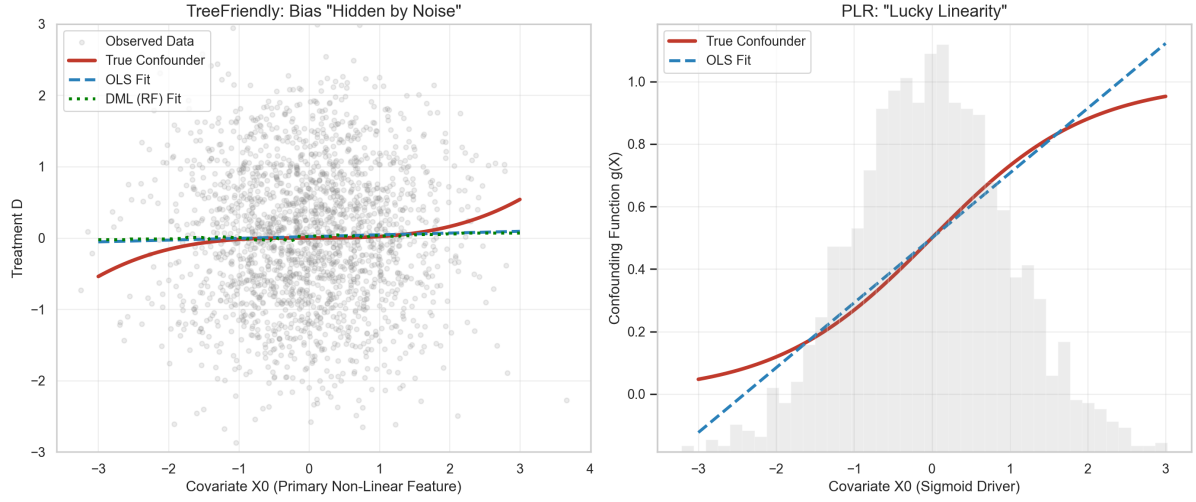


Figure 9.: Visualization of confounding structures and estimator behavior.

Note: (Left) TreeFriendly DGP: Displays the relationship between the treatment D and the primary non-linear covariate X_0 . The true cubic confounding function (Red) is obscured by high observational noise (Grey points), leading the linear OLS fit (Blue dashed) and DML (Green dotted) to miss the curvature. (Right) PLR DGP: Shows the true sigmoid confounding function (Red) overlaid with the covariate density (Grey histogram). The distribution of data is concentrated in the "linear" region of the sigmoid, allowing the misspecified linear OLS fit (Blue dashed) to approximate the true non-linear confounder with high accuracy, a phenomenon termed "Lucky Linearity."

A.7. Bias-Variance Decomposition

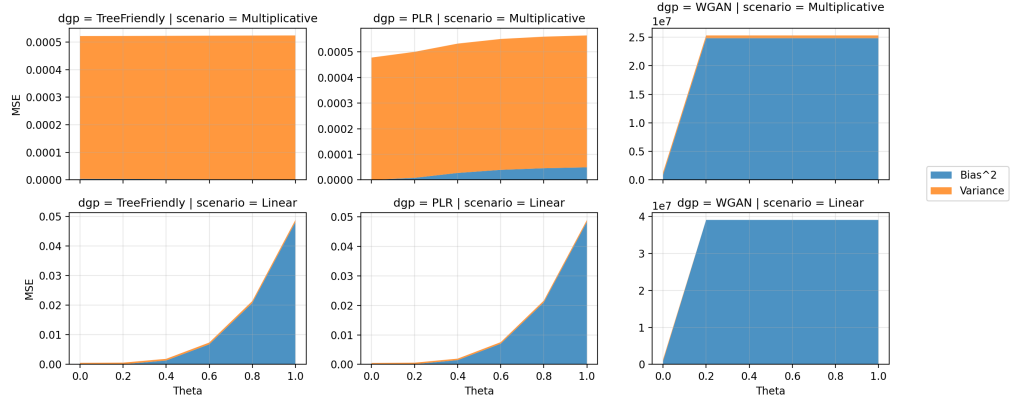


Figure 10.: Bias-Variance Decomposition: OLS

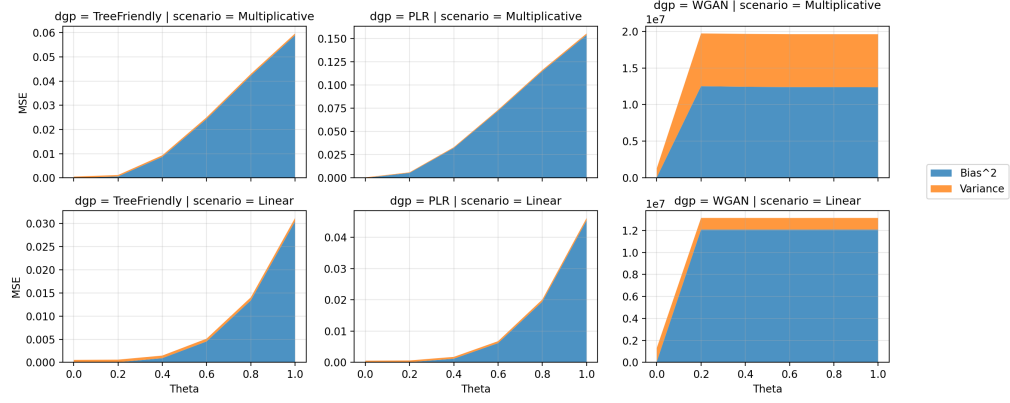


Figure 11.: Bias-Variance Decomposition: EconML

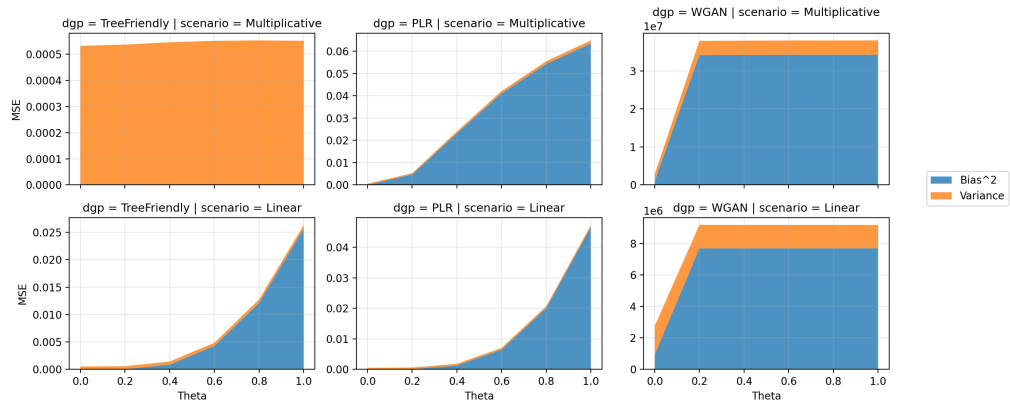


Figure 12.: Bias-Variance Decomposition: DoubleML

A.8. Estimator Performance Comparison

Table 2.: Comprehensive Estimator Performance Comparison with Relative Bias

DGP	Est.	Scenario	True ATE	Est. ATE	Bias	Rel. Bias	CI Width	Cov.
<i>Panel A: Baseline (Naive, Theta=0)</i>								
TreeFriendly	OLS	Naive	1.0	1.00	0.00	0.00	0.09	0.96
TreeFriendly	DoubleML	Naive	1.0	1.00	0.00	0.00	0.09	0.93
TreeFriendly	EconML	Naive	1.0	1.00	0.00	0.00	0.30	1.00
PLR	OLS	Naive	1.0	1.00	0.00	0.00	0.09	0.96
PLR	DoubleML	Naive	1.0	1.00	-0.00	-0.00	0.09	0.97
PLR	EconML	Naive	1.0	1.00	-0.00	-0.00	0.32	1.00
WGAN	OLS	Naive	6,250	7,223.67	972.72	0.16	2,728.75	0.75
WGAN	DoubleML	Naive	6,250	7,177.05	926.10	0.15	5,222.52	0.86
WGAN	EconML	Naive	6,250	6,448.78	197.83	0.03	23,818.77	1.00
<i>Panel B: Linear Collider</i>								
TreeFriendly	OLS	LinearCollider	1.0	0.78	-0.22	-0.22	0.11	0.00
TreeFriendly	DoubleML	LinearCollider	1.0	0.84	-0.16	-0.16	0.10	0.00
TreeFriendly	EconML	LinearCollider	1.0	0.82	-0.18	-0.18	0.35	0.53
PLR	OLS	LinearCollider	1.0	0.78	-0.22	-0.22	0.11	0.00
PLR	DoubleML	LinearCollider	1.0	0.79	-0.21	-0.21	0.11	0.00
PLR	EconML	LinearCollider	1.0	0.79	-0.21	-0.21	0.38	0.28
WGAN	OLS	LinearCollider	6,250	-1.68	-6,252.63	-1.00	0.68	0.00
WGAN	DoubleML	LinearCollider	6,250	3,404.69	-2,846.26	-0.46	4,671.30	0.29
WGAN	EconML	LinearCollider	6,250	2,735.42	-3,515.53	-0.56	22,499.78	1.00
<i>Panel C: Multiplicative Collider</i>								
TreeFriendly	OLS	Multiplicative	1.0	1.00	0.00	0.00	0.09	0.96
TreeFriendly	DoubleML	Multiplicative	1.0	1.00	0.00	0.00	0.09	0.93
TreeFriendly	EconML	Multiplicative	1.0	0.76	-0.24	-0.24	0.39	0.04
PLR	OLS	Multiplicative	1.0	0.99	-0.01	-0.01	0.09	0.97
PLR	DoubleML	Multiplicative	1.0	0.75	-0.25	-0.25	0.14	0.00
PLR	EconML	Multiplicative	1.0	0.61	-0.39	-0.39	0.65	0.04
WGAN	OLS	Multiplicative	6,250	1,275.74	-4,975.21	-0.80	2,695.27	0.00
WGAN	DoubleML	Multiplicative	6,250	297.19	-5,953.76	-0.95	9,389.84	0.23
WGAN	EconML	Multiplicative	6,250	2,524.47	-3,726.48	-0.60	69,681.68	1.00

Notes: **DGP** denotes the Data Generating Process. **ATE** is the Average Treatment Effect. **Relative Bias** is calculated as $(\text{Est. ATE} - \text{True ATE}) / \text{True ATE}$. **Coverage** represents the proportion of simulations where the 95% confidence interval contains the True ATE. The Linear Collider and Multiplicative Collider in Panel B and Panel C have strength of 1.0.